

School of Electrical and Computer Engineering, Purdue University

SoCET - System on-chip Extension Technologies

Non-blocking cache architecture for out-of-order processors

Vinay Jagan

Advisors:

Sooraj Chetput, Professor Mark Johnson

Table of Contents

Table of Contents	2
Overview	3
Report Overview	3
SoCET Overview	3
Architecture Background	4
The necessity of caches	4
Theory of caches	6
Cache parameters vs. miss rate	12
Out-of-order execution	13
Non-blocking caches	15
Architecture Design	16
System Overview	16
Parameterization and Banking	17
Miss status holding register (MSHR) buffer	19
Cache bank	23
Implementation and Verification Results	27
Test Sequences	27
Synthesis Results	27
Future Improvements	28
Pseudo/Tree-based LRU	28
Removing secondary miss coalescing	29
Teammate Contributions	30
References	31

Overview

Report Overview

The purpose of this project is to demonstrate work that has been done this semester in the completion of a non-blocking cache designed for out-of-order processors. This work was done with the premise that the architecture will be used for future SoCET projects that have a CPU/scheduling core implementation that utilizes out-of-order execution. This paper will highlight the necessity for this specific cache architecture, along with our design and implementation work explained in detail.

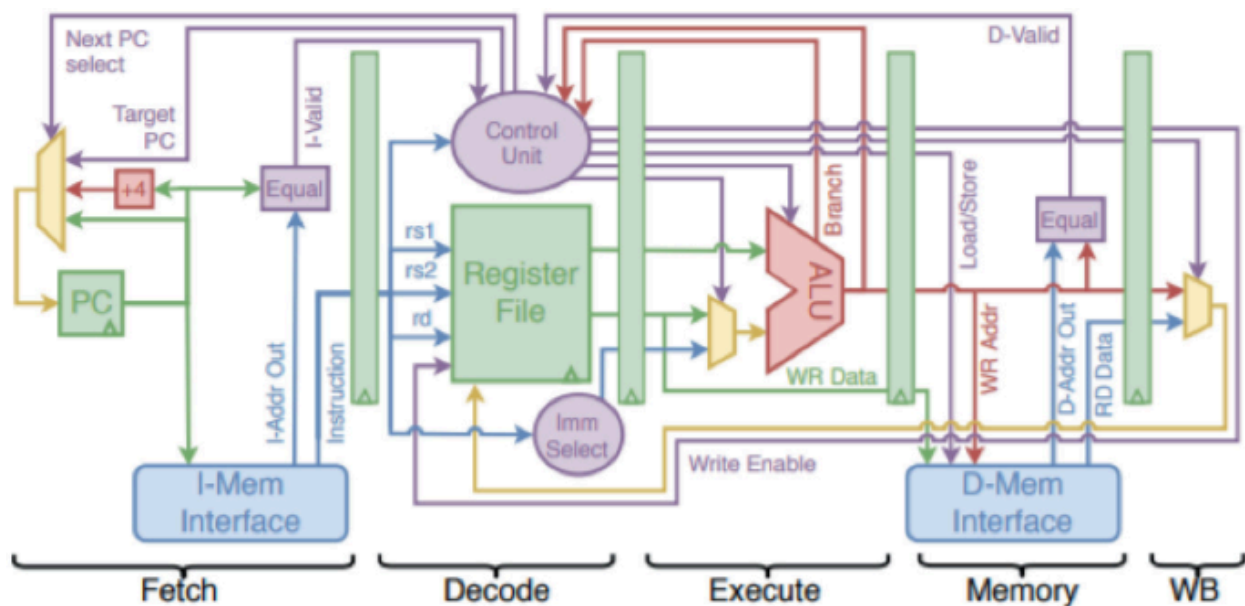
SoCET Overview

Purdue's System on-chip Extension Technologies team is a large cohort of undergraduate and graduate students in the Electrical and Computer Engineering that works on a large system on-chip module that includes numerous submodules such as a CPU, GPU, and AI processing unit. In addition to architecture-level implementation, a lot of work is also done for the full stack development of a chip, including design flow and mixed signal design. This project was developed within the AI Hardware subteam, with the expectation that the cache will be used for an out-of-order scheduler for an AI accelerator module. However, due to changes in team logistics and project designations, our implementation will be likely used for projects outside the subteam:

the most notable project is the data center CPU core that will be a superscalar, out-of-order processor.

Architecture Background

The necessity of caches



5-stage in-order processor [1]

To understand caches, one must look at a primitive 5-stage in-order CPU. The different stages enable faster higher clock frequencies by breaking up the processing of an instruction into stages, which minimizes critical path delay. A pipelined implementation also benefits in that instructions are processed consecutively even though its latency in cycles from read to completion is equal to the number of stages. Thus, the effective cycles per instruction (CPI) of a program is equal to or very close to 1. However, this

doesn't tend to be the case. An instruction at a specific stage may be forced to stall itself and the stages behind it because either a certain number of cycles have to be taken for computation purposes, or the stage has to wait for memory or I/O devices. These two reasons usually break down into either branching or memory accesses. For the sake of not detracting from the main purpose of this paper, let's focus on memory accesses.

Memory accesses involve reading and writing a system's DRAM. DRAM storage is a form of volatile memory that can be large, while relatively fast compared to disk access. Thus, a system with a lot of DRAM can perform memory accesses very quickly. However, from the perspective of a CPU, memory access takes 300-400 clock cycles, which means that completing instructions becomes very slow. When a memory instruction (load/store) enters the memory stage of an in-order processor, that instruction has to stall the entire pipeline for 300 cycles until the data access is complete.

A more formalized metric can be utilized to understand how memory accesses affect cycle times. Imagine that 30% of a program's instruction list consists of memory accesses, and that 70% is a combination of arithmetic and branching instructions that effectively complete in a single cycle (or no stalling). We can view the CPI of this theoretical program as: $1 + 0.3 \cdot \text{AMAT}$, where AMAT stands for average memory access time. Every DRAM access is about 300 cycles so our AMAT would be 300 cycles. This would make our CPI for 91. This is very disastrous for a CPU's execution time,

especially when a considerable amount of work is done to pipeline instructions to complete as fast as possible. Thus, a solution is necessary to improve average memory access time to improve execution time or CPI.

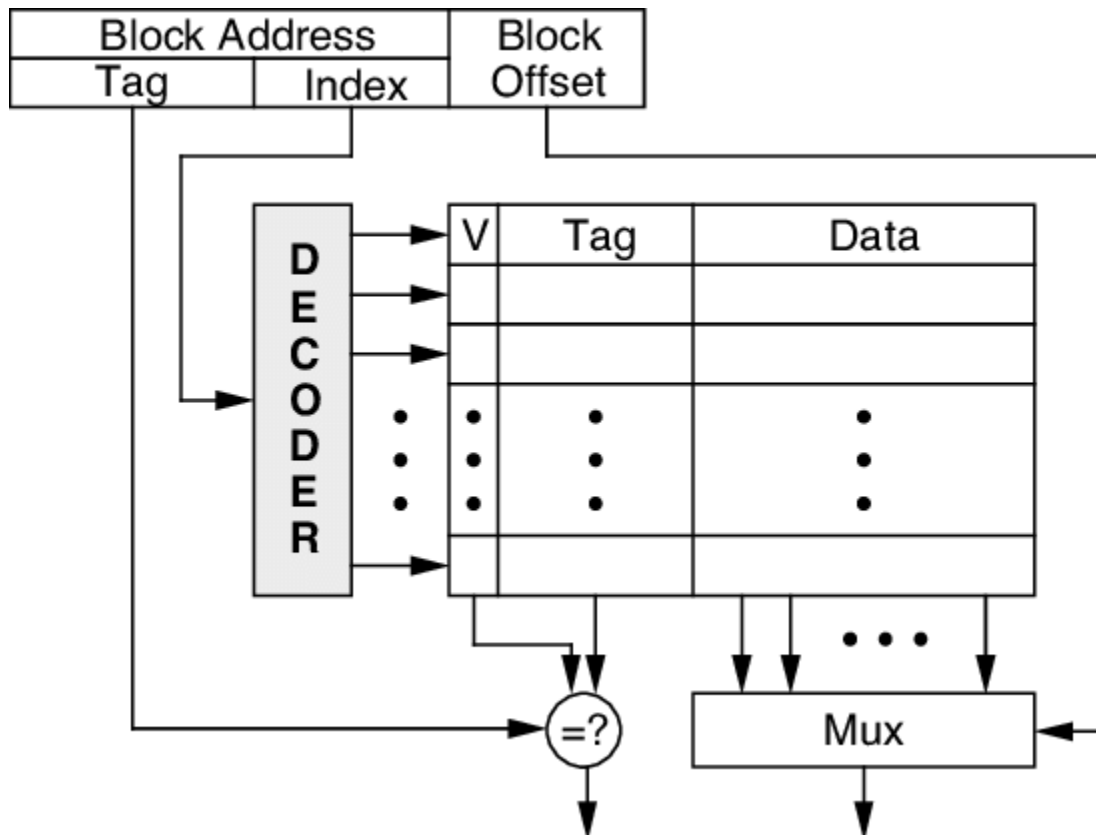
Theory of caches

A way to improve AMAT would be to utilize SRAM, which is a form of on-chip volatile memory that has faster access time (in the order of 2-3 cycles). However, not all of our memory can fit on SRAM. A standard SRAM cell is significantly larger than a DRAM cell and has much higher power usage. Thus, if SRAM were to be used, it could only represent a subset of the total address space. However, memory accesses could represent the entire address space of our DRAM, potentially higher when considering virtualization. Thus, if on-chip SRAM existed, it must somehow hold data that the CPU would need immediately.

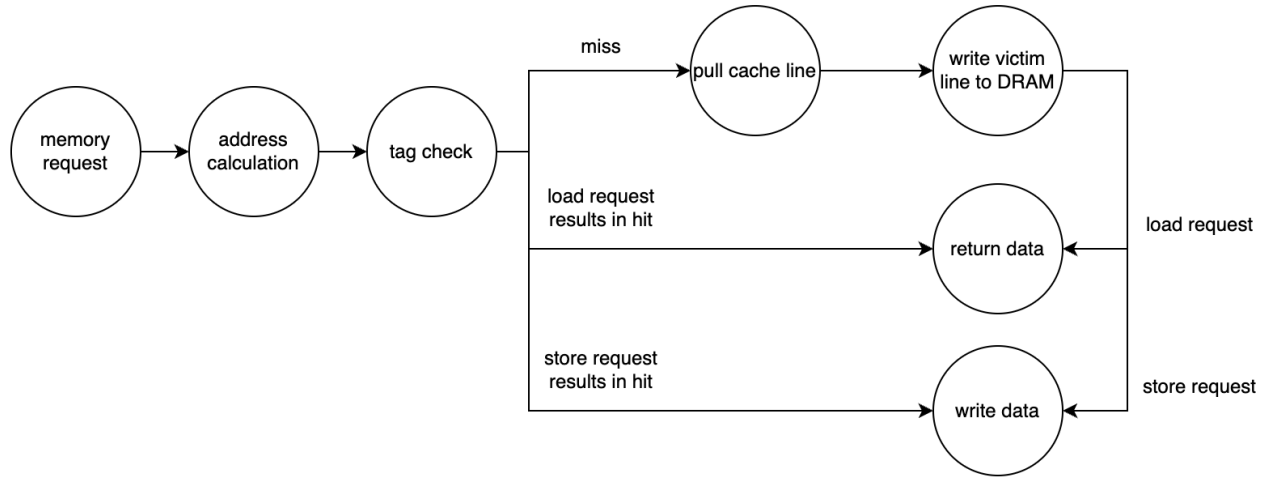
Luckily, there are two software phenomena that exist that enable on-chip memory to be useful: spatial and temporal locality. Spatial locality dictates that data access from consecutive load/stores are likely to use addresses that are next to or nearby each other. For example, a program with an array is likely to use nearby memory addresses since arrays are stored consecutively. An even better example is pulling instructions from DRAM, since consecutive instructions have consecutive memory addresses, with the exception of branch/jump instructions. The other phenomena is temporal locality

that claims that a specific data access is likely to be re-used within a program's execution.

A cache is an architectural solution to improve AMAT by exploiting spatial and temporal locality. In essence, it is a collection of SRAM cells that stands in between the CPU and memory controller. Upon a memory access, the cache can either service data it holds, or send a request to the memory controller to obtain the data from DRAM.



Standard cache arrangement [2]



Operation sequence of a general-purpose cache

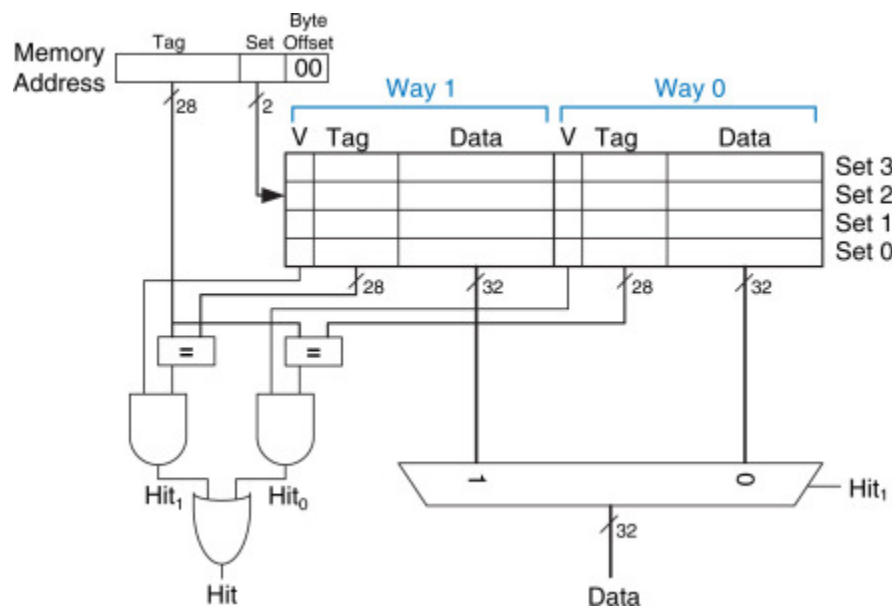
Caches are usually arranged in an array of cache lines, where each line holds a valid bit, a dirty bit, a tag value, and a data array. When a load/store enters the cache, the cache determines whether or not the requested address exists in the cache by first determining the index of the cache line, and then comparing the tag (as well as the validity of the line) to determine if the address matches. If both checks pass, then a cache hit is presumed, meaning that the requested data access is already in the cache. The cache can then index the block (data array) to obtain the specific value or write to it. Upon every cache write, the dirty bit for that line gets turned on, indicating a modified value that is not present in DRAM, but present in the cache. The opposite of this is a cache miss, in which the correct line does not exist already. If the valid bit is off, it is simply a matter of pulling in the new block, writing it into the data array, and servicing the load/store. However, if a line exists previously at that index, the cache must check the dirty bit if it is modified and then eject that previous line back out to DRAM. It is

important that only dirty lines are capable of being ejected as victims because important cycle time would be lost ejecting data that DRAM already has.

Determining the correct line and validating the block is determined by masking the address to find the index and tag value. Usually, the block length and number of lines are predetermined, so it is simply a matter of using powers of two to determine the necessary bits and their locations within the address to find the block offset and line index. The tag itself is composed of the bits that are left over starting from the most significant bit. This also demonstrates that a primitive cache can have roughly two parameters: the overall cache size and the block length (usually in number of words). The number of lines can be derived by dividing the cache size by the block length.

Misses are generally classified into three forms: compulsory, capacity, or conflict. A compulsory miss is when a block address (address of word 0 in a block) is first seen by a program, which means that a miss is inevitable because the cache will never have seen that block before. A capacity miss is when the cache gets too filled that another access that is sequential will have its index wrap around back to the top of the cache and evict used lines. A conflict miss is when two addresses are accessed that are not close to each other, but map to the same index, resulting in the previously used line being evicted. While compulsory misses are unavoidable, capacity and conflict misses can be mitigated to improve the overall cache miss rate. Reducing capacity misses is simple: increase the size of your cache; however, reducing conflict misses involves using set-associativity.

As discussed before, conflict misses occur when addresses with varying tags have the same index. The most recent access will eventually replace that line. This means that if the previous line ever gets used it leads to another conflict miss. A hardware solution to this would be to increase the number of lines that get stored in each index of the cache. Thus, the cache gets arranged into sets that contain a certain number of ways (cache lines) all indexed together. This concept is called set-associativity and is critical for reducing conflict misses.



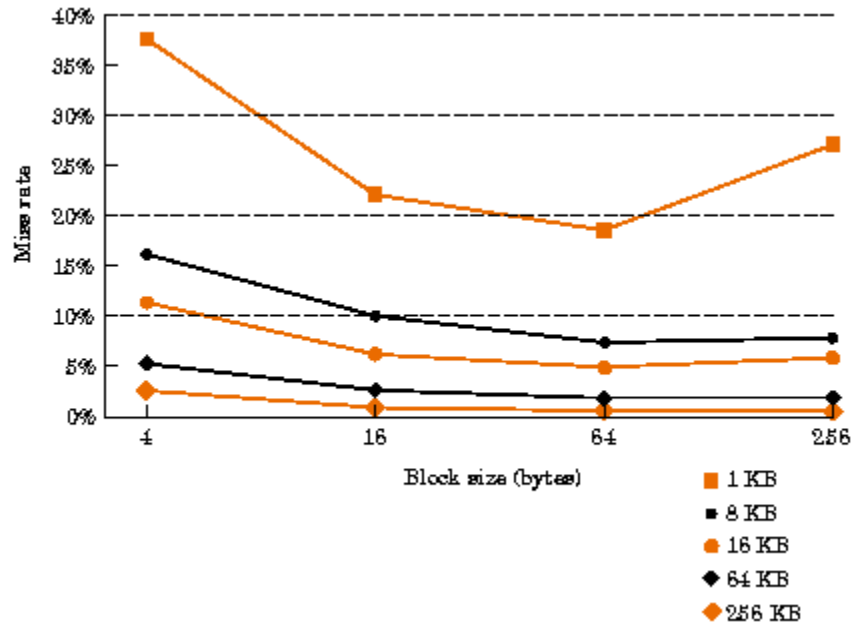
Set-associative cache block diagram [3]

While this technique does reduce conflict misses, it does also increase access time from the cache since the added logic to compare tags across all ways adds to the critical path delay. However, the lower miss rate may account for an overall decrease in CPI or max clock frequency: if there is a large access time, an architect could choose to adjust the entire max clock frequency, or have pipelined stages for address calculation

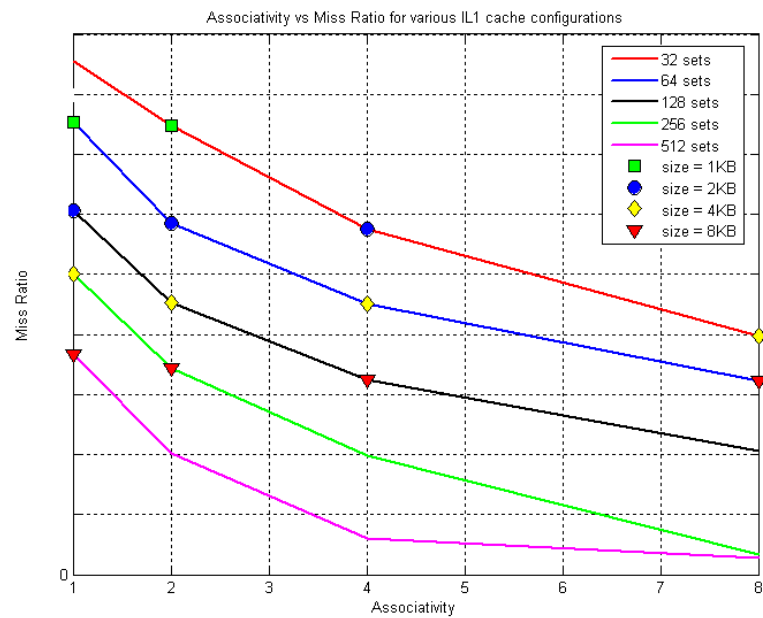
and tag check across sets. An extensive analysis using software profiling and Amdahl's Law would be necessary in determining whether the lower AMAT would benefit more than increased cycle time/count. Generally, most modern processors tend to include higher associativity for L1 caches.

With associativity, however, comes with more design choices. On top of choosing the level of associativity (i.e. number of lines/frames within a set), one must choose the writeback policy. Even with more frames per index, it is possible that a cache line within a set may be filled, which would mean that a line would need to be ejected to make room for another block's data. Thus, a decision needs to be made as to which cache line gets ejected given that a block address could map to any possible line in a given set. The algorithm to choose a frame in a set to replace is called a replacement policy. The most optimal approach is to identify the line that will be used the furthest from the present (Belady's algorithm). However, knowing the future is impossible. It could, however, be approximated using different algorithms. Some of the algorithms include First-In First-Out (FIFO), Most Recently Used (MRU), and Least Recently Used (LRU). The closest to optimal algorithm is LRU, and is generally the most used. LRU works by applying an age to each frame in a set. A frame that is most recently used (either just pulled into the cache, or a hit access) will have the smallest age. The frame with the largest age will be chosen to be ejected if necessary.

Cache parameters vs. miss rate



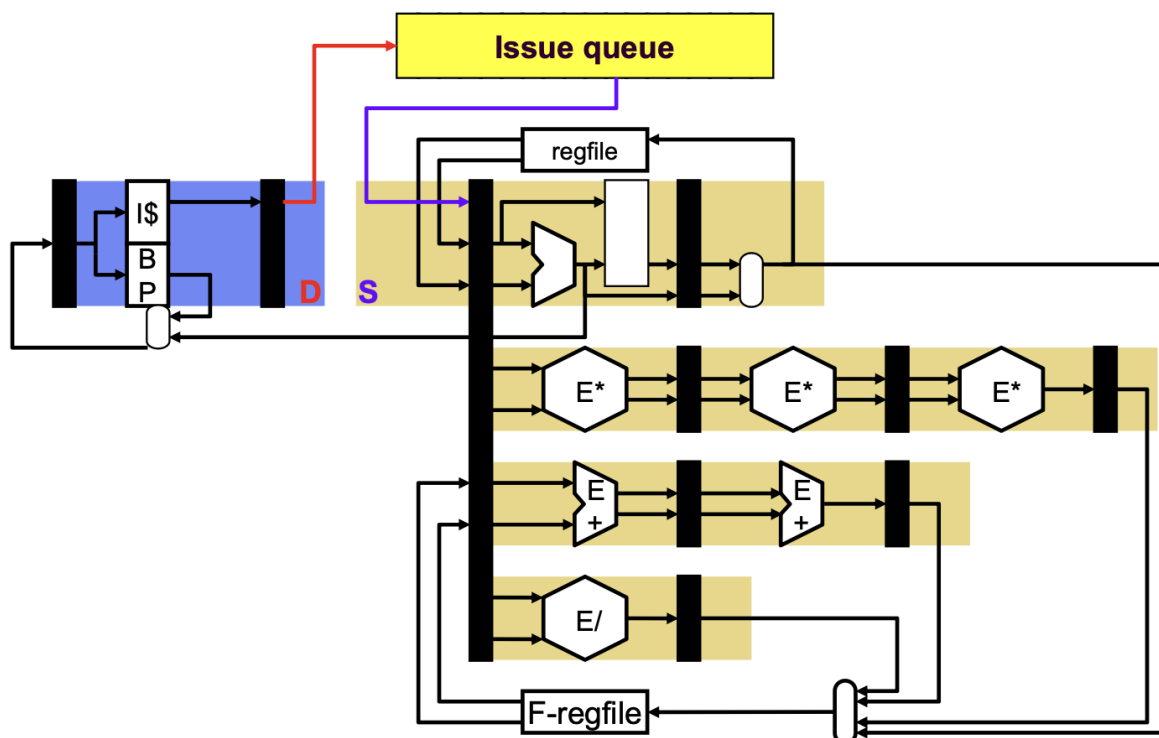
Miss rates by varying block sizes [4]



Miss rates by varying cache sizes and associativity [5]

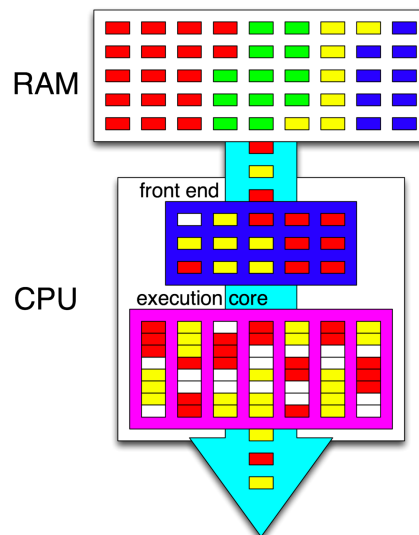
Out-of-order execution

Out-of-order execution is a method employed by modern processors to improve performance by executing instructions as soon as their input operands become available, rather than strictly following the sequence specified by the program. By allowing instructions that are ready to execute to proceed ahead of others that are waiting for resources or data dependencies to resolve, processors can maximize instruction-level parallelism and significantly reduce idle time. This approach enables more efficient utilization of processing units, resulting in improved throughput and reduced execution latency.



Superscalar, out-of-order processor example [6]

The figure above is an example of an out-of-order processor. While the load/store unit is not directly shown, it is also another functional unit in which instructions can be dispatched to. Its use and structure plays a critical role when attempting to use the processor for simultaneous hyperthreading (SMT). SMT enables multiple threads to use the superscalar aspect of a CPU by coexisting together in its dispatch units. This allows memory accesses to be overlapped such that multiple cores are all being efficiently used at once. However, for this to occur the CPU cannot be stalled when processing memory accesses.



Execution layout for a SMT core [7]

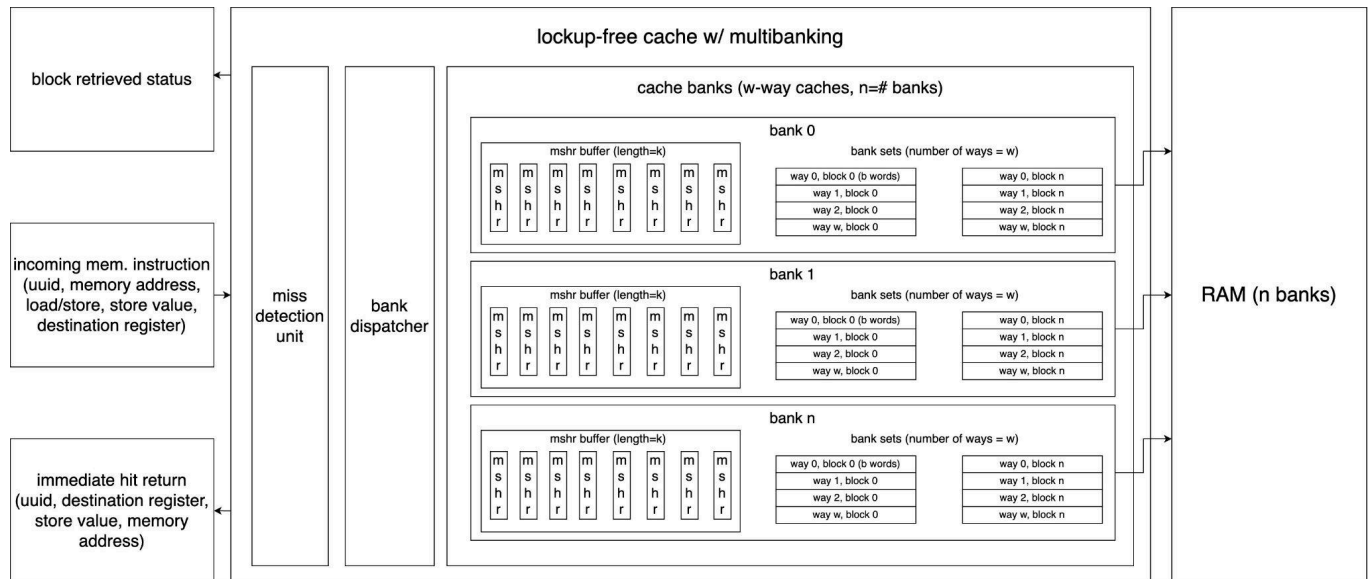
Non-blocking caches

A non-blocking cache enables the ability of the CPU to not be stalled by a cache miss. The non-blocking cache usually will allow misses to enter a buffer and then perform a miss at a time. Simultaneously, a memory access that is discovered to be a hit, can be handled alongside a miss. Both a buffer and parallel miss/hit capability enable the CPU to continue to do work with a low chance of being stalled. While this may conceptually seem not complex, there is a significant amount of corner cases and adjustments that have to be considered.

The very first decision that needs to be considered is how the CPU handles a miss. While we know that the non-blocking cache must store this miss in a buffer, how does the CPU know to not stall, as well as know what miss is being returned back to them? For the time being, let's work under the assumption that a hit is determined and returned in a single cycle. In this case, when a hit is not returned upon the input of a memory instruction, the CPU can assume that its instruction has entered the miss buffer. From the cache, a UUID will be generated that is unique to that memory instruction and provided to the processor for safe-keeping. When a miss is finished, the UUID will be returned along with the address and potentially load data, indicating that the instruction is completed and ready for the writeback stage, if necessary. This, along with other design decisions, enable complete, and most importantly, correct sequencing of operations that limit structural hazards and minimize stalling.

Architecture Design

System Overview



Full system block diagram

The full system block diagram shown above demonstrates the overall design that was implemented. The core cache structure is encapsulated within a series of banks that holds a subset of the total cache size. The main reasons for this is to minimize long path delays to access each set, as well as take advantage of multi-banked DRAM access, if provided. Incoming memory instructions enter the module, and within the same cycle gets released if a hit is detected. If not, the instruction enters a miss status holding register (MSHR) buffer, where each register holds the block address as well as the potential store data. The buffer acts like a FIFO, where the head element is latched onto

its complementary bank to be processed as a miss. When the miss completes, the bank that processed it, sends a UUID corresponding to that memory instruction. A hit works in parallel where the incoming memory instruction to the overall module gets indexed to the correct bank and the access is performed, all within one cycle. One could notice from the diagram that in any given clock cycle, up to N (number of banks) misses and one hit can be operated on simultaneously.

Parameterization and Banking

A primitive cache with associativity can be established with the following parameters: cache size, block length, number of ways. The number of total sets can then be derived using these parameters. Our implementation of a non-blocking cache, however, has multiple banks and a buffer within each bank, which adds two extra parameters: number of banks and MSHR buffer length.

Parameters:

- Total cache size (bytes) = C
- Block length (words) = L
- Number of ways = W
- Number of banks = N
- MSHR buffer length = K

Number of sets: $C / (L * W)$

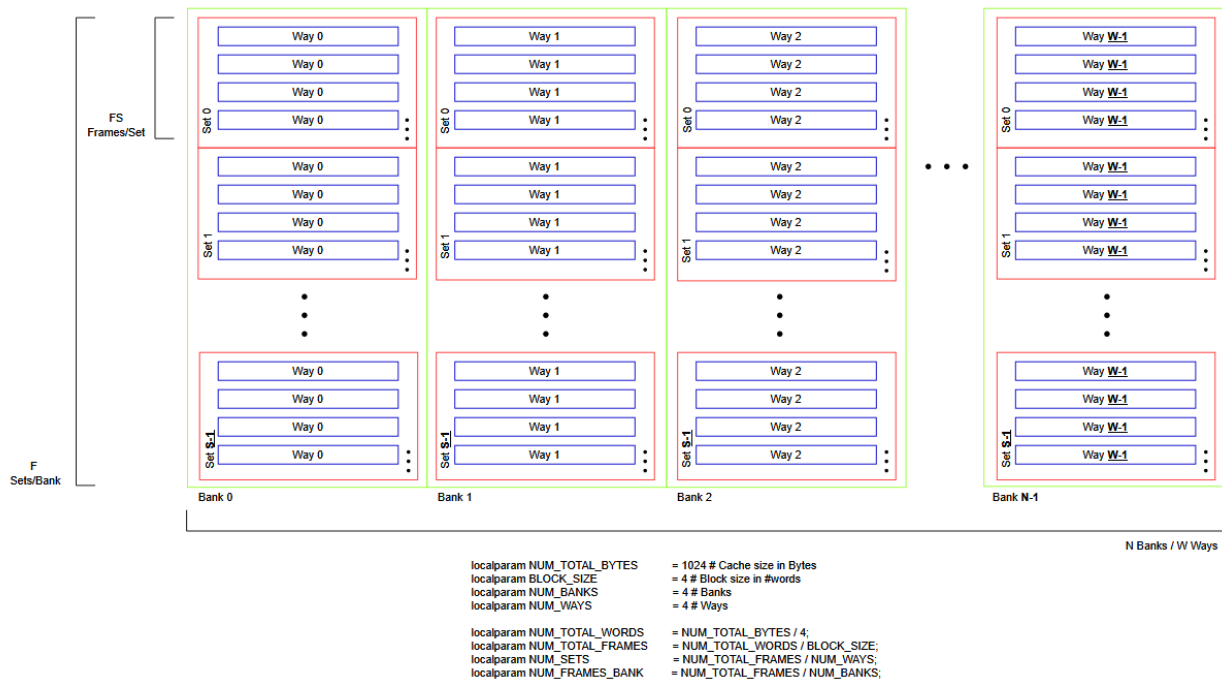
Number of sets per bank: $C / (L * W * N)$

Set index = (address >> (log2(L) + 2))[log2(Number of sets) - 1:0]

Bank index = set index % N = set index & (N - 1)

Set index in bank = set index / N = set index >> log2(N)

Parameterization calculations



Bank Layout

Each bank contains a subset of the total number of sets in the entire cache. To maximize interleaving in a way that takes advantage of spatial locality, consecutive set indices are in different banks. For example, if a sequence of memory accesses uses 4 consecutive words, but our cache only holds 2 words per block, then two separate, but consecutive banks will access the two blocks in parallel. The figure directly above this paragraph highlights how the set indexing works with our banking structure, given that the overall set index essentially increments. In other words, the overall set index modulus the number of banks determines which bank to place the set in. The overall set index when integer divided by the number of banks is used to determine at which index within a bank's specific cache structure should that set be in. The overall set index, as described in the background, is determined from the address itself.

Miss status holding register (MSHR) buffer

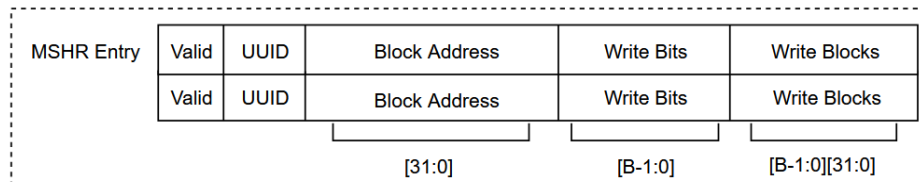
When an incoming memory instruction is detected to not be a hit on the cycle in which it has entered, a miss is to be presumed. On a miss, the system determines which bank the address must enter and places it in the MSHR buffer of that bank. The buffer works like a First-In First-Out (queue) data structure where the first element that enters must be the one that first exists. The buffer is a bit more simplified than general hardware implementations.

An incoming miss is first wrapped into a miss status holding register. The register holds a valid bit, the block address of that miss, a series of write bits and a copy of the block that gets put into the cache. The write bits are a sequence of bits where the length is equal to the number of words in a block. Upon a store miss, a specific word in that block address is meant to be filled and that block offset is turned on in the bit array, as well a copy of the data to be written in the correct position of the block copy. With this, each MSHR represents a block address that has a series of words to be placed at specified indices. The benefit of having each MSHR only represent a block address is that subsequent misses of the same block address but different word address can be combined into the MSHR that already exists in the buffer. This concept is called secondary miss coalescing where an already existing MSHR with the same block address (primary miss) of the incoming miss (secondary miss) can be combined such that both misses are represented with a single MSHR. Upon coalescing, a secondary miss that is a store turns on its offset of the write bit array and writes its data to the block

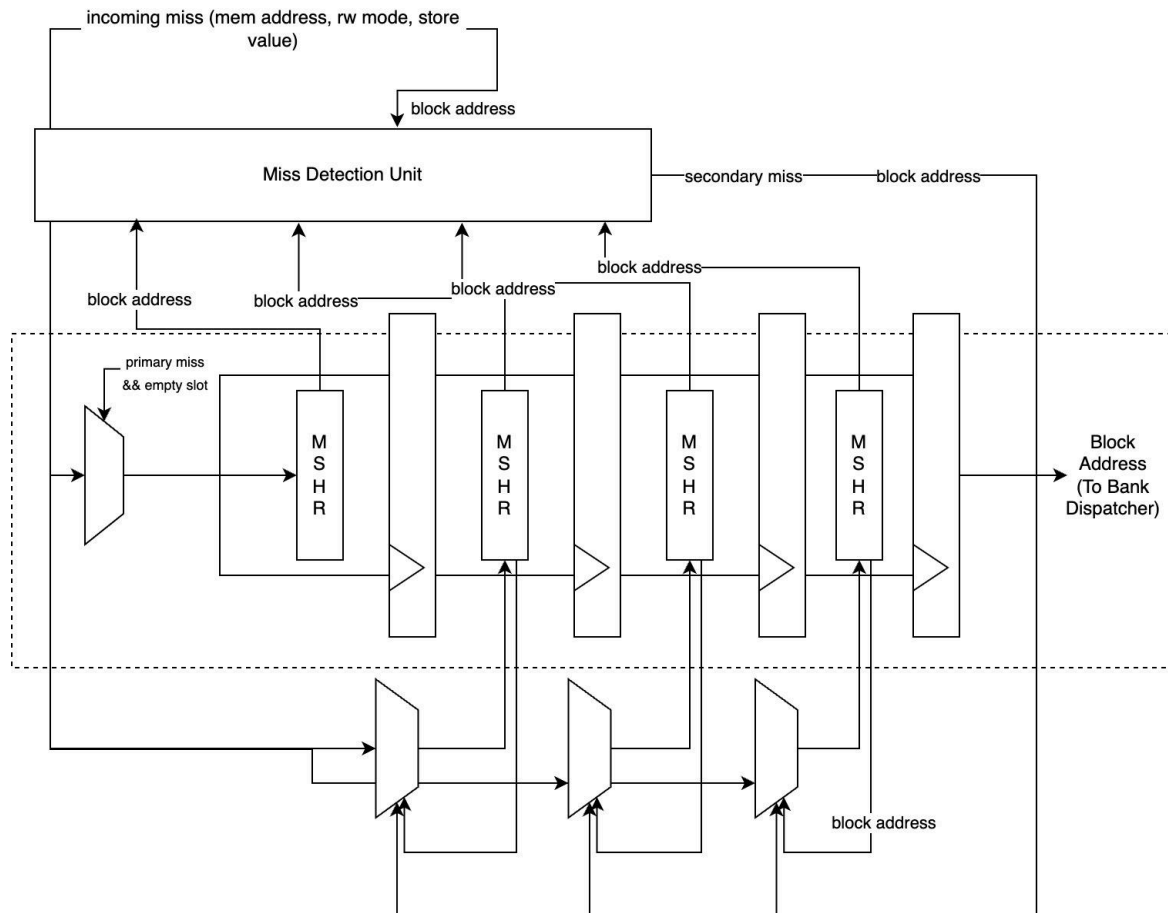
copy at the correct offset. The benefit of coalescing is that fewer cycles are used to process subsequent misses within the same block address, which tends to be a common case, especially in the context of arrays or other packed data structures. According to Amdahl's Law, handling this case is extremely important given that its frequency of occurrence is high.

The buffer itself works with the following sequence of operations. Firstly, the head of the buffer is latched into the actual cache bank, meaning that the MSHR of that slot cannot be modified. When the bank is processing the MSHR, a stall signal is indicated to the buffer to not remove the element at its head. When a new miss enters the buffer, it first attempts to coalesce on a primary miss. If no primary miss is detected, a new MSHR is created to be placed into the buffer, and a UUID for the instruction is generated. Each element is placed in the very last index of the buffer, but can only be placed if the last slot is either empty (valid bit is off) or the stall signal is clear. Every other element shifts forward if the slot ahead of them is empty or the stall signal is off. This differs from common approaches to queue-like data structures in hardware since instead of a two-pointer approach, there is essentially a fixed pipeline of slots and a new element can only be entered into the end, or combined with an existing slot. This does imply that a new miss in an empty buffer will not be immediately processed: the new MSHR has to wait a few cycles as it propagates from the tail to the head of the buffer. While this may seem costly, it is important to note that every element in the MSHR is a miss that may take in the order of a few hundred cycles to be processed; thus, there is not much of a difference in cycles to make a noticeable effect.

An interesting corner case arrives with this logic: a MSHR may enter the bank but should be resolved as a hit. Assume that the cycle delay between a primary miss and secondary miss entering a bank is greater than the length of the buffer. The primary miss will already be latched into the bank when the secondary miss enters, meaning that the secondary miss actually has to enter in a new slot since the head of the buffer cannot be modified. The solution that we implemented for this was a MSHR hit detection that will be explained in more detail in the bank architecture section.



An individual miss status holding register



MSHR buffer RTL diagram

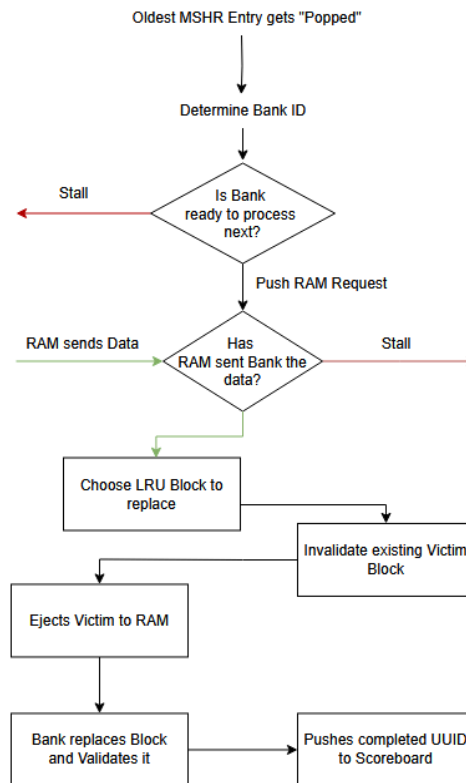
Cache bank

The actual cache part of the bank works very similar to the operations of a primitive cache upon a hit. A separate input port exists for the memory instruction that was just given by the processor. The bank first checks if the memory instruction belongs to this bank, and then compares the tags across all lines of the requested set. If there is a tag match, the hit is processed within the same cycle. If a hit is not detected, then that instruction is expected to re-enter the bank as a MSHR. Another input port exists for the head of the MSHR slot. This slot is expected to be a miss and enters a state machine that pulls the correct block from the cache, except for the blocks that are meant to be rewritten according to the write bits. Before the block gets pulled, however, a victim block gets determined, if necessary, and its valid bit is turned off. This is to ensure that a hit that arrives will not be able to modify the victim. The victim itself is found using a LRU replacement policy. After the block gets pulled, the victim gets ejected and the LRU updates.

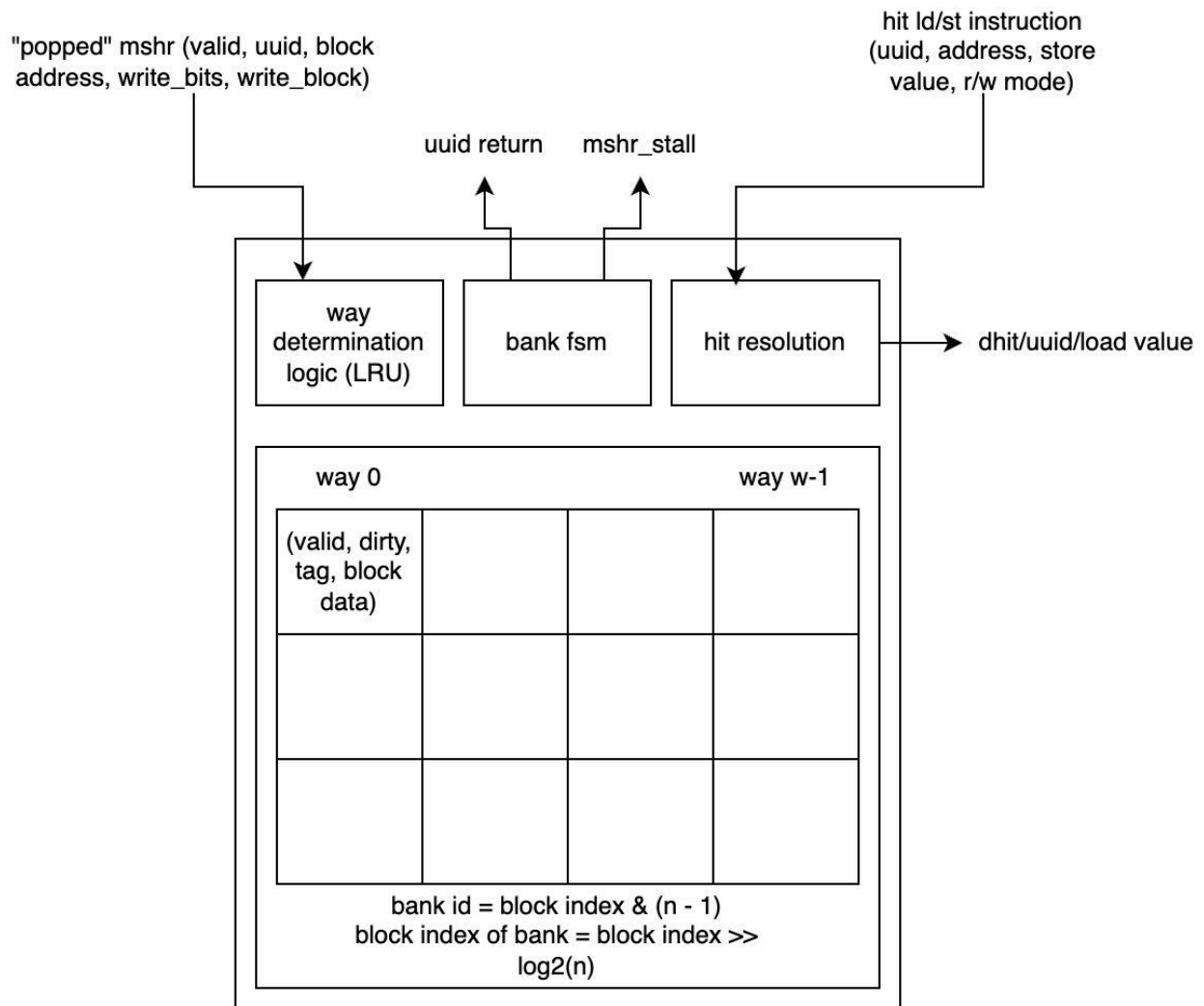
Implementing LRU in hardware for a two-way set associative cache is simple, but for parameterized associativity it can be quite challenging. While a linked-list approach is generally conceived of in software, re-arranging blocks is difficult. To implement LRU, we created an age data structure that holds a value for the age of each block. When a block in a set either has a hit or a miss attached to it, its age becomes zero and the rest of the blocks age increments by 1. At every clock cycle, the LRU block in each set gets

calculated and stored such that the next access knows which block to use with calculating ahead of time.

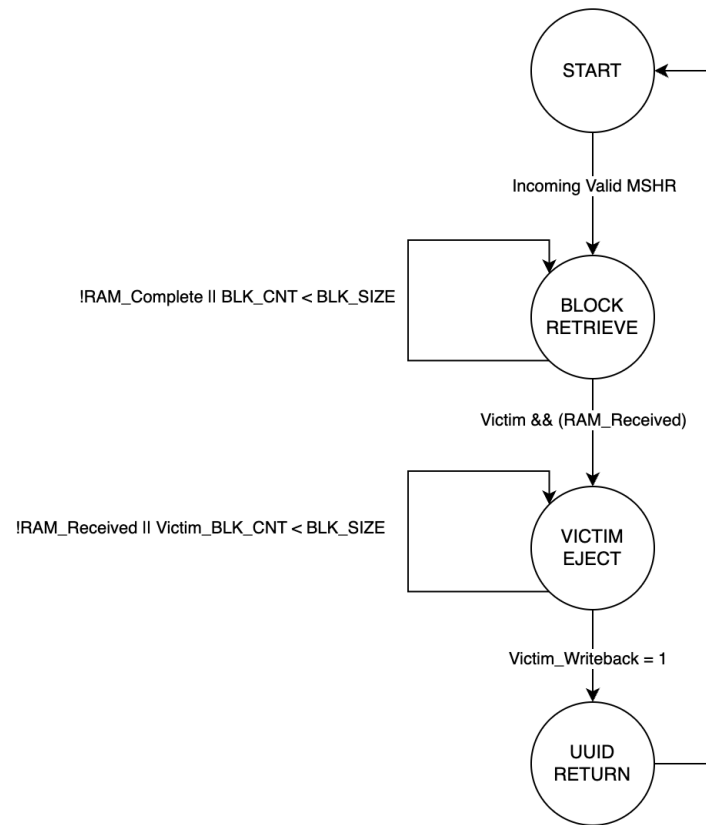
There is an important corner case that was mentioned in the MSHR buffer section where a miss may functionally be a hit even though it arrived through the MSHR slot since a secondary miss may not coalesce a primary miss that is latched onto the bank. To resolve this issue, we perform a hit check on MSHRs that alerts a latched signal informing the state machine to not pull blocks, but allow writes to them to occur, given that the MSHR contains it.



Bank operation sequence



Bank block diagram



Bank FSM

Implementation and Verification Results

The non-blocking cache was fully implemented as specified above. Each individual module was test benched, as well as the overall architecture. All test cases listed below passed successfully.

Test Sequences

- Secondary miss coalescing
- Hit check
- LRU replacement
- Secondary miss corner case
- Basic read/writes

Synthesis Results

The system was also fully synthesized as well on the MIT-LL 90nm PDK targeted for 1GHz. Our design had a few major critical paths with the largest being the LRU calculation.

```
Path 1: VIOLATED (-553 ps) Setup Check with Pir lru_reg[0][lru_way][0]/CLK->D
View: view_1p2_25
Group: clock1
Startpoint: (F) mem_instr_in[addr][index][0]
Clock: (R) clock1
Endpoint: (F) lru_reg[0][lru_way][0]/D
Clock: (R) clock1
```

Synthesis results

The resulting clock frequency was 643MHz, which suggests that there is a lot of room for improvement, especially since the team's desire is to have a 1GHz chip.

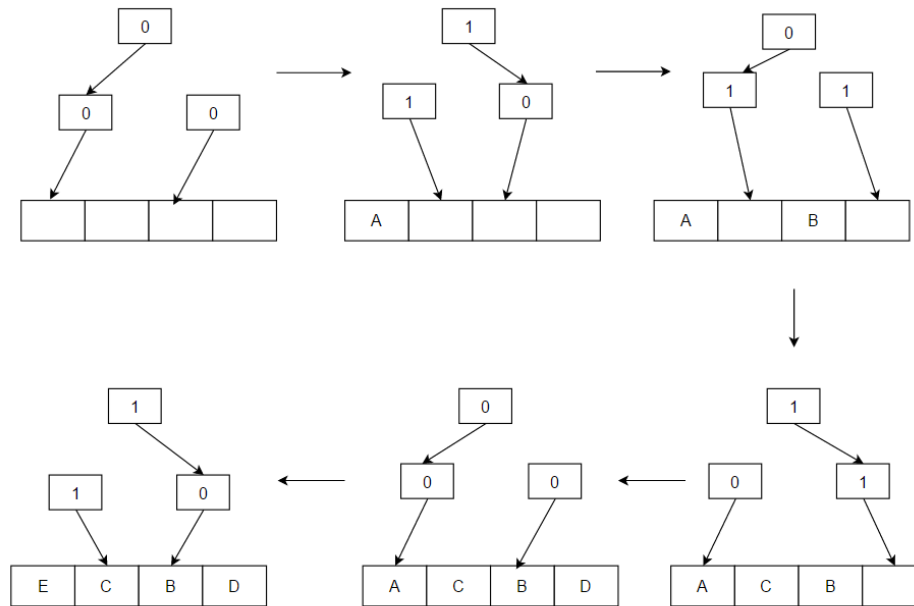
Future Improvements

Since we didn't achieve our target frequency, we need to pipeline a lot of aspects of our design and possibly change some functionality to be a lot more minimal or less complex.

Pseudo/Tree-based LRU

One such change could be to our hardware implementation of LRU. Right now, there is a long cascade of gates since each way's LRU has to be compared with the rest to eventually finalize on the correct block. Since this all happens in one cycle, the critical path is very long. To fix this, we have to use a more heuristic method for LRU that uses less bits and doesn't compare large numbers.

The tree-based LRU algorithm essentially has a tree of pointers that direct towards a single block that is the LRU. When the LRU has to change, the list of nodes that start from the root to the block all flip states to represent the next block.



Tree-based LRU

This implementation greatly minimizes critical path and uses less gates overall.

Removing secondary miss coalescing

Secondary miss coalescing is also a large critical path violation. There is no easy way to resolve this other than removing it. The reason why it is such a long sequence of wires is because we are comparing block addresses across all elements of the buffer. Thus, it may be best if we simply remove secondary miss coalescing. Thankfully, our bank can resolve MSHR hits without pulling data from DRAM, so while there is an extra overhead of cycles, it may not be as grave as one would assume.

Teammate Contributions

Vinay: MSHR buffer, Top-level

Akshath: Bank

References

[1]

https://media.springernature.com/lw685/springer-static/image/chp%3A10.1007%2F978-981-19-0098-3_73/MediaObjects/520266_1_En_73_Fig4_HTML.png

[2]

<https://www.researchgate.net/publication/3800962/figure/fig1/AS:669950486274049@1536740051086/Block-diagram-of-a-direct-mapped-cache.png>

[3]

<https://ars.els-cdn.com/content/image/3-s2.0-B9780128200643000088-f08-09-9780128200643.jpg>

[4] ECE 565 slides

[5]

https://upload.wikimedia.org/wikipedia/commons/thumb/2/27/Hyper-threaded_CPU.png/1200px-Hyper-threaded_CPU.png

[6] <https://www.cs.fsu.edu/~hawkes/cda3101lects/chap7/F7.12.gif>

[7] <https://www.ecs.umass.edu/ece/koren/architecture/SimpleScalar/lab1cachesfig1.gif>